

Comprehensive Guide to EEG Schizophrenia Classification with Deep Learning

A Technical Walkthrough of Neurological Sensory Gating and PyTorch CNN Pipelines

Author:	hrijit123
Date:	May 2026
Language:	Python 3.11 / PyTorch 2.12
Framework:	Deep Learning / Convolutional Neural Networks

1. Neurological and Medical Background

Schizophrenia is a severe chronic neurological and psychiatric disorder affecting approximately 1% of the global population. It manifests through positive symptoms (such as auditory hallucinations and delusions), negative symptoms (such as social withdrawal and flat affect), and cognitive deficits. To understand how artificial intelligence can identify Schizophrenia from brainwaves, we must examine a clinical deficit known as **Sensory Gating**.

Sensory Gating and the Corollary Discharge Mechanism

The human brain is constantly bombarded with sensory input. To function efficiently, the brain filters out redundant or self-generated stimuli. This filtering mechanism is called sensory gating. Sensory gating is regulated by a neurological mechanism known as **Corollary Discharge** (or *efference copy*).

When you initiate an action—such as preparing to speak, moving a limb, or pressing a button—your brain's motor regions send a copy of that command to the sensory cortexes. This efference copy signals the sensory cortex to expect feedback, causing it to suppress its response. This is why you cannot tickle yourself: your brain predicts the tactile feedback and dampens the neural response. In patients with Schizophrenia, the corollary discharge pathway is impaired. The brain fails to flag self-generated sensations as 'self', leading to sensory over-stimulation. This failure is neurologically linked to auditory hallucinations, as patients hear their own internal speech but fail to recognize it as self-generated, perceiving it as external voices.

Event-Related Potentials (ERPs) and the N100 Wave

The sensory gating mechanism is measured using Electroencephalography (EEG) during a **Button-Tone task**. Subjects are exposed to two main conditions: pressing a button that immediately triggers a sound (self-generated tone), and passively listening to the same sound (externally-generated tone).

During this task, researchers track the **N100 wave** (a negative deflection in the electrical potential of the auditory cortex peaking approximately 100 milliseconds after the stimulus is presented). In healthy control subjects, the N100 wave is significantly suppressed when they press the button to trigger the tone themselves, compared to when they hear it passively. In Schizophrenia patients, there is little to no suppression of the N100 wave during the button-press tone task. This difference in electrical potential serves as the neurological biomarker that our deep learning models learn to classify.

2. Data Processing and Feature Engineering

The clinical dataset consists of EEG recordings from 81 subjects. Each subject has a file containing multiple trials. The processing pipeline transforms these raw signals into clean, normalized tensors that are optimized for neural network training.

Downsampling via Row-Averaging

Each EEG trial consists of 70 electrode channels recorded over 9216 time steps. Feeding a matrix of shape (9216, 70) directly into a neural network creates a massive feature space (645,120 features) which causes overfitting and high memory consumption. To resolve this, we downsample the signal by averaging consecutive rows in blocks of 16:

Timesteps Downsampled = $9216 / 16 = 576$ timesteps

This row-averaging acts as a temporal low-pass filter, dampening high-frequency noise while preserving the low-frequency Event-Related Potential (ERP) components like the N100 wave.

Max Normalization per Electrode Channel

Because scalp thickness, skull density, and contact impedance vary between subjects, EEG signal amplitudes differ. To establish a standardized feature representation, we apply max normalization independently to each electrode channel. This scales the raw microvolt readings into a range of [-1.0, 1.0], stabilizing the model gradients and speeding up convergence.

Local Caching (.npz)

Parsing 81 massive CSV files, looping through trials, and running matrix math on every execution is extremely slow. To optimize the workflow, the dataset module compresses and caches the processed arrays into a single NumPy file: data/preprocessed_eeg.npz. Subsequent runs load from this cache, starting training in under a second.

3. Deep Learning Model Architectures

This project implements three distinct PyTorch convolutional neural networks, each representing a different design philosophy for interpreting EEG signals.

Original 2D CNN (OriginalEEG2D)

This architecture treats the EEG trial as a spatial-temporal grid (like a single-channel image) of shape (1, 70, 576), where the height represents the 70 electrode channels and the width represents the 576 time points. It uses 2D convolutions with a rectangular kernel size of (5, 20). This wide filter convolved across time and channels extracts spatial interactions (how sensors behave relative to neighbors) and temporal patterns simultaneously. It uses a **Tanh** activation function to match the original Keras model, followed by MaxPool2D layers, dropout, and a linear output logit layer.

1D CNN (EEG1D)

A 1D CNN treats the 70 electrodes as input features (similar to RGB channels in an image) and performs 1D convolutions solely over the 576 time steps. By restricting the convolutions to the time axis, the model captures temporal waveforms (like the shape of the N100 wave) independently for each sensor. This approach is highly efficient, uses fewer parameters, and respects the time-series nature of EEG signals.

EEGNet Architecture

EEGNet is a specialized, compact CNN developed for Brain-Computer Interfaces (BCIs). It consists of three primary convolutional stages:

1. **Temporal Convolution:** Performs a 2D convolution over time with a wide kernel, capturing frequency-domain bands (e.g., alpha, beta, theta waves).
2. **Depthwise Spatial Convolution:** Convolves across all 70 channels for each temporal filter individually, learning spatial brain maps (which brain regions are active).
3. **Separable Convolution:** Combines spatial and temporal features using point-wise convolutions, minimizing model size while maximizing representative power.

4. Code Walkthrough and Training Mechanics

How the Model Learns

The model is trained as a binary classifier using **BCEWithLogitsLoss** (Binary Cross Entropy with Logits). The model outputs a raw unbounded real number (logit). The loss function applies a Sigmoid activation to calculate the probability of Schizophrenia (ranging from 0.0 to 1.0) and computes the loss relative to the actual label. By calculating logits directly instead of using a sigmoid activation inside the model, we prevent vanishing/exploding gradients during training.

Dataset Module Code Snippet (dataset.py)

The downsampling and normalization logic are implemented in numpy as follows:

```
def average_rows(matrix, n):
    shape = matrix.shape
    # Reshape into blocks of n rows
    reshaped = matrix.reshape(shape[0] // n, n, shape[1])
    return reshaped.mean(axis=1)

# Normalize electrode columns (max norm = 1)
X_resaped = X.reshape(-1, num_electrodes)
X_norm = normalize(X_resaped, axis=0, norm='max')
X_norm = X_norm.reshape(X.shape)
```

Original 2D CNN Model Implementation (model.py)

Here is the PyTorch implementation of the original CNN-2D model with dynamic flattening:

```
class OriginalEEG2D(nn.Module):
    def __init__(self, num_channels=70, timesteps=576):
        super(OriginalEEG2D, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=(5, 20))
        self.pool1 = nn.MaxPool2d(kernel_size=(5, 15))
        self.conv2 = nn.Conv2d(32, 13, kernel_size=(3, 3))
        self.pool2 = nn.MaxPool2d(kernel_size=(3, 3))
        self.dropout = nn.Dropout(0.2)
        self.flatten_dim = self._get_flatten_dim()
        self.fc1 = nn.Linear(self.flatten_dim, 317)
        self.fc2 = nn.Linear(317, 1)

    def forward(self, x):
        x = x.view(-1, 1, self.num_channels, self.timesteps)
        x = self.pool1(F.tanh(self.conv1(x)))
        x = self.pool2(F.tanh(self.conv2(x)))
        x = self.dropout(x).view(x.size(0), -1)
        return self.fc2(F.relu(self.fc1(x)))
```